

VAT Identifier Discovery

NOTE: This document serves the sole purpose of illustrating my results and documented observations regarding the VAT Identifier Discovery Task for the Data Assets Intern role at Veridion. I should note that I used LLM tools to optimize code and brainstorm solutions, but every aspect written below has been personally checked, tested, and validated.

Part 1: Research & Discovery Trails

1. Domain Discovery: The Missing Link

The Companies House bulk dataset provides reliable baseline identifiers (Company Name, Number, Status) but lacks a crucial data point for open-web extraction: the company's official website. Without knowing the domain, crawling for VAT numbers is impossible.

2. DuckDuckGo Search Results & Limitations

Initial Approach: I built a programmatic search module using the duckduckgo-search Python library to query "[Company Name] UK official website", bypassing aggregators (Companies House, Endole, CompanyCheck, LinkedIn) via an IGNORE_DOMAINS filter, with artificial delays to avoid HTTP 429 rate limiting.

Execution: Ran on a micro-sample of 20 active companies from Companies House.

Result: Non-aggregator websites surfaced for 5 of 20 companies (25% apparent yield). Open-web search via free APIs is a high-latency, low-yield approach — it works for established businesses but leaves a massive gap in coverage for local SMEs that operate purely via social media or have no dedicated web presence at all.

3. Targeted Scraping & Regex Extraction

Extraction Logic: UK VAT numbers follow a strict 9-digit structural constraint. I implemented the regular expression

```
(?i)(?:VAT|GB)[^0-9]{0,15}?(?:[0-9]{3}[\s\-]?[0-9]{4}[\s\-]?[0-9]{2})
```

designed to capture 9-digit sequences in proximity to keywords like "VAT" or "GB", tolerant of arbitrary whitespace or hyphenation as commonly found in page footers. Note: this pattern was not paired with the Mod-97 checksum in the PoC crawler itself — see Part 1.9 for why that filter matters and where it belongs in production.

4. The SEO Pollution Dead End

The Expectation: Strict query parameters ("{Company Name}" UK official website) plus an aggregator blacklist would yield genuine corporate homepages for active companies.

The Evidence: On manual inspection of the 5 apparently "successful" domain discoveries, the crawler had actually surfaced microsoft.com, merriam-webster.com, zhihu.com, and amazon.de — none of them the target company's site.

Why it failed: When a small UK company (e.g. "GUIDE AND MENTOR LIMITED") lacks a dedicated domain, the search engine falls back to keyword matching against ultra-high-authority domains (the word "guide" matching a Merriam-Webster dictionary page). This is a catastrophic false-positive class, not a coverage gap: it renders downstream scraping worse than useless, because it produces confident-looking wrong answers rather than honest "not found" results.

5. Methodology Refinement: Known-Good Data Injection

The Problem: Programmatic discovery yielded 0% functional, non-aggregator websites once the 5 apparent hits were manually checked. Continuing with this sample alone would prevent testing the downstream extraction and validation modules at all.

The Decision (Transparency): I made an explicit, documented decision to inject two known-good entities — Waterstones Booksellers Limited and British Council — into `sample_with_domains.csv`, purely to test end-to-end integration of the extraction and HMRC validation logic. This is a standard rapid-prototyping technique for exercising a pipeline when an upstream stage has failed, and it is disclosed here rather than presented as an organic result.

Edge-Case Highlight: The British Council operates under a Royal Charter and, as a charity, does not carry a standard Companies House number. Dummy 8-digit placeholders were injected into the `CompanyNumber` column to satisfy the CSV schema. This surfaces a real production issue: rigid dependency on the Companies House schema as the sole foundational identifier will fail to capture charities, royal charters, and foreign branches that hold valid UK VAT registrations but don't fit the standard company-number shape.

6. Anti-Bot & Architecture Limits

The Execution: Ran the regex-based crawler against the two known-good, high-transparency entities.

The Result: 0% extraction success on both.

- **Architecture limits:** British Council's international landing page does not surface a UK-specific tax identifier — that data sits several clicks deep in a legal/terms subpage. A homepage-only crawler cannot reach it.
- **Waterstones — cause not fully isolated:** the scraper already sends a realistic Chrome desktop User-Agent header, so the 0% result cannot be attributed to a missing browser fingerprint on its own. The exception handler in the PoC catches broadly and logs only the exception type, not the HTTP status code or response body, so the exact cause wasn't captured at run time. Two explanations remain open: active bot defense (WAF/Cloudflare) returning a non-200 response, or — at least as likely — the VAT number sitting in a footer that's rendered client-side via JavaScript, which a `requests + BeautifulSoup` fetch never executes, so the text simply isn't present in the HTML the scraper receives. These point to different fixes (proxy/fingerprint rotation vs. headless rendering), so this is flagged as an open question rather than a settled diagnosis.

Conclusion for production: Regardless of which cause dominates, a production crawler cannot rely on a single GET to the index page. At minimum it needs headless-browser rendering (Playwright/Puppeteer) to execute client-side content and survive anti-bot challenges, plus response-level logging (status codes, not just exception types) to tell the two failure modes apart — and, more fundamentally, a different pipeline shape entirely (see 1.9 below).

7. Environment Architecture & Dependency Management

The execution environment is containerized with Docker (`python:3.11-slim`) for reproducibility across machines. Third-party dependencies (`pandas`, `requests`, `beautifulsoup4`) are pinned in `requirements.txt`; core operations that don't need them (JSON parsing, regex matching) use only Python's standard library to keep the image lightweight. Note on the bulk file: the 2GB Companies House CSV is read with `usecols` to load only the four needed columns into memory, not with chunksize-based streaming — a real production version would need proper chunked or out-of-core reads (e.g. `Dask`, or `pandas chunksize`) rather than this PoC's single-pass, column-pruned read.

8. Data Validation Strategy

The verification module calls the official HMRC Developer API (api.service.hmrc.gov.uk) directly rather than scraping the public "Check a UK VAT number" web interface. The web UI is fragile and liable to break on any DOM change; the API returns a structured, deterministic JSON response and is the correct integration point for anything meant to run unattended.

9. Alternative Deterministic Routes (Identified but Untested)

Given how badly the domain-first route performed, a production system should not extend it — it should replace its core shape. These routes were identified during research but not implemented or tested in this PoC; they are listed here as findings, not results.

- **Inverting the flow via Common Crawl:** instead of the brittle Company → Domain → VAT chain, process the Common Crawl .co.uk corpus directly with a parallelized regex pass, apply a local Mod-97 checksum filter to the candidates, validate the survivors against the HMRC API (which returns the registered name and address), and reverse-join that confirmed output back onto Companies House by name/address matching. This removes domain discovery — the step that failed — from the critical path entirely.
- **The EORI shortcut:** a UK VAT-registered company's EORI number is, with very few exceptions, GB + [9-digit VAT number] + 000. EORI numbers surface in customs documentation, logistics/freight registries, and some public shipping records, giving a second, structurally deterministic route to the same identifier.
- **Public spending disclosures (Contracts Finder, The Gazette):** local councils and central government are required to publish supplier spend, and some of these disclosures link vendor names directly to VAT numbers in structured form — no scraping required, though coverage is limited to suppliers who sell to the public sector.
- **The Mod-97 filter vs. brute force:** the space of mathematically valid UK VAT numbers is not 10^9 but on the order of 10^7 , because the Mod-97 (and 97-55) check-digit algorithms reject most 9-digit strings outright. Generating all valid numbers and querying HMRC for each would be technically capable of reconstructing most of the live registry for free — and is exactly why it should not be done: it is mass enumeration of a government database rather than lookup, it violates the spirit and likely the letter of HMRC's API terms, and it would trigger rate limiting and credential bans long before completion. The checksum's legitimate role is as a local pre-filter on candidates already found some other way, cutting API calls spent on regex noise before they're made — never as a discovery method in its own right.

Part 2: Proof of Concept — Execution & Metrics

The PoC implements each pipeline stage — domain discovery, VAT extraction, HMRC validation — as a working, independently runnable module, but the stages are not wired together end to end in the code: 05_hmrc_validator.py was run against two hardcoded values (a known-real Waterstones VAT number and one invented number) to confirm the HMRC integration works correctly, rather than consuming 04's output file directly, since 04 produced no extracted VAT numbers to consume. The metrics below are reported as measured, including where the result is zero.

Metric	Value
Initial sample	20 active companies, drawn from Companies House
Apparent domains surfaced (DuckDuckGo)	5
Valid corporate domains (after manual check)	0
Known-good entities injected for pipeline testing	2 (Waterstones, British Council)
VAT numbers extracted	0
VAT numbers confirmed by HMRC	0
False-positive rate	N/A — denominator is zero

Conclusion on metrics: An accurately measured null result is valuable engineering information; an unmeasured metric is not. The 0% success rate on this sample and route empirically demonstrates that open-web NLP-based domain discovery, chained to shallow HTTP scraping, is the wrong architecture for this problem at national scale — not that the underlying data doesn't exist on the open web (see 1.9 and Part 3).

Part 3: Scaling, Cost & Data Integrity Monitoring

1. Infrastructure & Bottlenecks

A direct-crawling architecture requires investment in residential proxy networks to get past CDN/WAF defenses, but the real ceiling is the HMRC API's rate limit, not bandwidth. A clustered architecture (e.g. Kubernetes running Celery/RabbitMQ-style async workers) would manage proxy rotation, retries, and exponential backoff against that limit, and would need to queue candidate validation independently of the discovery/crawling stage so one doesn't block the other.

2. Cost Estimation per Company

HMRC does not publish an official daily quota for the "Check a UK VAT number" endpoint; the figure below is an assumption built from third-party integrations that report throttling themselves to roughly 3 requests/second to stay within HMRC's upstream limits, stated here explicitly as an estimate rather than a documented fact.

Validation capacity: ~3 req/s per credential set → in practice, allowing for backoff and off-peak scheduling, roughly 50,000–100,000 validations/day is a realistic sustained figure per credential set, comfortably below the theoretical ceiling of ~259,000/day.

Discovery cost (the larger driver): processing the Common Crawl .co.uk segment (a few TB of compressed WARC/WET data) for regex candidates is a batch Spark/EMR-style job — order of a few hundred to low-thousands of USD in compute per full pass, depending on corpus scope and refresh frequency.

Scale math: against 4.2M active companies, and assuming a conservative 10–20% yield of extractable, checksum-valid candidates (consistent with the ~25% raw domain-discovery yield observed in this PoC,

before that yield collapsed under manual inspection), that implies roughly 400,000–800,000 HMRC validations — achievable within about two weeks running a small number of credential sets in parallel.

Bottom line: amortized over compute, storage, and pipeline maintenance, marginal acquisition cost lands at low cents per successfully validated company. The constraint is not infrastructure spend — it's discovery yield and legal/ToS exposure, both of which cost engineering time and risk tolerance, not dollars.

3. Measuring Accuracy Without Ground Truth

- **Confidence scoring:** fuzzy-match (Levenshtein distance) the company name returned by HMRC against the name in Companies House. A $\geq 95\%$ text match is treated as near-certain confirmation of a correct join.
- **Conflict-rate monitoring:** flag any case where the same VAT number is assigned to two distinct Companies House entities, or multiple different VATs surface for a single domain. A spike in this rate is an immediate signal of extraction regression, independent of any external ground truth.
- **HMRC validation yield:** track the daily ratio of extracted candidates to successful HMRC 200-OK responses. A drop indicates the regex extractor has started picking up noise — phone numbers, product codes, or other 9-digit sequences masquerading as VAT numbers.

Debate Topics

1. Brute-forcing the HMRC checker via checksum

Covered in detail in Part 1.9: technically capable of reconstructing most of the registry for free, but a bad idea in production — it's mass enumeration rather than lookup, breaches HMRC's API terms, and would trigger bans long before completion. The checksum's correct role is as a local pre-filter on candidates already sourced some other way, not as a discovery method.

2. Keeping the dataset current

Companies register and deregister continuously, so a static snapshot goes stale quickly. Production architecture would consume Companies House's real-time streaming API for incorporation/dissolution events, and apply a TTL (e.g. 30–90 days) to previously validated VAT numbers, requeuing them for re-validation against HMRC as they approach expiry — rather than re-processing the entire dataset on a fixed batch schedule.

3. Detecting dataset corruption without ground truth

With nothing complete to compare against, correctness has to be inferred from internal signals rather than an external reference: confidence scoring via name-matching against Companies House, conflict-rate tracking for duplicate or contradictory VAT assignments, and HMRC validation-yield monitoring to catch regex drift. None of these prove correctness individually, but together they catch the failure modes that matter — silent misassignment and gradual extractor decay — without needing a labeled dataset.

4. Sources not comfortable selling in a commercial product

The clearest example from this PoC itself: scraping DuckDuckGo's search results for domain discovery. It works around a search engine's own terms of service rather than using a licensed data source, and on top of the legal exposure it produced unreliable results — the false-positive domain mappings documented in Part 1.4. I would not ship that as part of a paid dataset. The same reasoning extends to anything that requires circumventing access controls or that surfaces unverified personal data: the legal and reputational risk outweighs the coverage gained.